

We are excited to assess you for the **Software Engineer Instructor** role. This assessment evaluates your development skills through two focused, real-world exercises that reflect the dynamic and multifaceted environment at **Uncommon.org**.

You are expected to demonstrate clean, well-structured, and well-tested code, along with clear technical reasoning. Unless a part specifies otherwise, you may use the **programming language, framework, and tools of your choice**.

You have **3 working days** to complete the assessment. **Automated tests** are expected for the coding exercise to ensure the robustness and reliability of your work.

You must create a **GitHub, Bitbucket, or GitLab repository** and share it upon completion. Your repository should display all commits with meaningful messages reflecting your progress.

Assessment Instructions

This assessment has **two parts**, and you are required to complete **both**. Together they give us a rounded picture of how you build, test, and reason about software.

- **Part 1 — Coding Exercise:** assesses hands-on implementation, edge case handling, and testing discipline.
- **Part 2 — System Design:** assesses architecture, tradeoff reasoning, and technical communication.

Part 1: Coding Exercise — In-Memory Rate Limiter

Project Goal:

Build a reusable **rate limiter** that protects an internal API from abuse by deciding whether a given request should be allowed or rejected, demonstrating correctness, clean design, and strong testing habits.

Requirements

1. Sliding-Window Policy

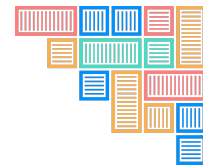
- Allow at most **N** requests per **W** seconds, per client key (e.g. an API key or user ID).
- Different keys are limited independently.

2. Clean Interface

- Expose a simple method, e.g. **allow(key)** returning true (allowed) or false (rejected / throttled).

3. Testability

- Make the current time injectable so behaviour is deterministic and easy to test.



4. Automated Tests

- Cover the core behaviour, including window boundaries, the first request, key isolation, and burst traffic.

Constraints

- In-memory only — no external datastore such as Redis. However, structure the code so a distributed backend *could* be swapped in later.
- You may assume a single process for the core exercise, but note what would change under concurrency.

Deliverables

- Source code and automated tests in the repository.
- A short **README** (3–6 sentences) describing your approach, key tradeoffs, and what you would do next with more time.

Optional Bonus

- Add a second interchangeable policy (e.g. a **token bucket**) behind the same interface.
- Make the limiter thread-safe and add a test that exercises concurrent access.
- Add cleanup so idle keys do not leak memory over time.

Part 2: System Design — Notification Service

Project Goal:

Produce a **system design** for a notification service used by a product with millions of users. Other internal services call it to send notifications, which it delivers across **email, SMS, and push**. We are assessing your architecture, your reasoning about tradeoffs, and how clearly you communicate technical decisions.

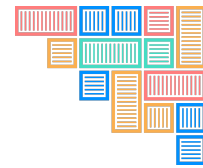
Requirements to Address

1. Core Functionality

- Support multiple channels (email, SMS, push) and remain extensible to new ones.
- Respect user preferences (opt-outs, per-channel settings, quiet hours).
- De-duplicate so the same notification is never sent twice.
- Track delivery status (sent / delivered / failed).

2. Scale & Reliability

- Handle roughly 10 million notifications per day, including large spikes such as a marketing blast.



- Guarantee no lost notifications — at-least-once delivery with idempotency.
- Keep transactional notifications (e.g. a password reset) near real-time, while bulk traffic may be delayed.
- Tolerate unreliable, rate-limited third-party providers (e.g. email/SMS/push vendors).

What to Submit

- A written design document (Markdown or PDF) in the same repository, including a clear architecture diagram.
- Explanation of your major components, data stores, and the tradeoffs behind each choice.
- How you handle retries, failover, de-duplication, and prioritisation of transactional vs. bulk traffic.

Shortlisted candidates may be asked to walk through this design in a short follow-up discussion.

General Notes

- This assessment evaluates your **technical skills, attention to detail, and ability to write secure, maintainable, well-tested software**.
- We value clear reasoning as much as working code — explain the tradeoffs behind your decisions.
- You are encouraged to add **extra features or depth** to showcase your competency and creativity.

Good luck!

Submission Details

- **Deadline:** 26 Aug 2026
- **Share:** repository link (with Part 1 code and the Part 2 design document) to elishanorman78@gmail.com CC elisha.urombo@uncommon.org CC quinton@uncommon.org

We look forward to reviewing your submission!